MULTI-THREADED DESIGN FOR TEXT RETRIEVAL

Kanishk Gorana
S20240030378

Kartik Kumthe Sanjay
S20240030383

Manas Jain
S20240030387

***Abstract*— Efficient text retrieval across large bilingual corpora requires balancing parallel throughput against thread synchronization and memory overhead. This paper presents a high-performance multithreaded architecture implemented in C and POSIX Threads (Pthreads) for extracting targeted English–Hindi sentence pairs from a 1.03-million-sentence subset of the IIT Bombay parallel corpus. We evaluate algorithmic improvements spanning naive sequential scanning, 1D data partitioning, byte-balanced block decomposition, and two-phase bilingual candidate filtering. To prevent concurrency bottlenecks, our design couples a shared read-only 64-bit byte-offset index with thread-isolated file descriptors, private line buffers, and lock-free thread-local result vectors. Benchmarks across 1 to 16 threads demonstrate substantial latency reductions, decreasing search times from 1.922 s to 0.405 s (4.75 times speedup) for government and from 1.072 s to 0.333 s (3.22times speedup) for the. In controlled multi-run evaluations, 16-thread search kernel latencies reach approximately 0.135 s (5.6times speedup). However, parallel efficiency declines from 100% to under 30% at 16 threads. Profiling reveals that this scaling limit is governed by Amdahl's Law and specific threading overheads: thread lifecycle management, memory bus bandwidth saturation during concurrent streaming, workload imbalance from non-uniform sentence lengths, and serial result-merging costs. Furthermore, two-phase candidate filtering provides an algorithmic breakthrough by converting full second-language scans into targeted index seeks, reducing secondary verification work from O(N) to O(|K|) candidate matches.**

Keywords— Multithreaded retrieval, bilingual corpus, Pthreads, parallel processing, byte-offset indexing, workload balancing, word frequency, co-occurrence, cache hierarchy.

I. INTRODUCTION

Parallel corpora provide sentence-level cross-lingual alignments essential for machine translation, bilingual lexicon induction, and multilingual natural language processing. The IIT Bombay English–Hindi Parallel Corpus [1] is widely utilized for these tasks. However, training and evaluating specialized models frequently requires extracting domain-specific or vocabulary-constrained "mini-datasets" rather than reprocessing the entire multi-gigabyte corpus for every downstream task. To support targeted retrieval, our system evaluates query predicates spanning monolingual tokens (English or Hindi), bilingual boolean combinations (AND/OR), and multi-word vocabulary sets, while ensuring strict sentence alignment and duplicate elimination.In unindexed environments where building and maintaining persistent inverted postings lists is impractical or resource-prohibitive, retrieval relies on scanning the raw text. While sequential scanning over millions of records incurs prohibitive latency, multithreading provides an avenue to accelerate throughput. In this work, we present a high-performance multithreaded text retrieval architecture developed in C using POSIX Threads (Pthreads), evaluated on a quality-filtered subset of 1,030,875 aligned English–Hindi sentence pairs (score> 0.65) from the IIT Bombay corpus.

A. Our system addresses key systems and algorithmic challenges in multicore text processing:

1. Algorithmic Progression Beyond Brute-Force Scanning: While 1D data partitioning divides sequential work across p workers, bilingual AND queries enable algorithmic pruning. By introducing a two-phase retrieval pipeline, the system scans the primary language in parallel to generate candidate sentence IDs, then performs targeted O(1) random seeks via a lightweight 64-bit byte-offset index on the secondary language. This reduces secondary verification from an exhaustive O(N) full scan to O(|K|) candidate evaluations.
2. Lock-Free Concurrency and Memory Isolation: To avoid severe synchronization bottlenecks during high-frequency matching, the architecture couples a shared, read-only offset table with thread-isolated FILE* file streams, independent line buffers, and private dynamic result buffers, eliminating mutex contention in the critical matching path.
3. Mitigating Workload Imbalance: Natural language text exhibits high variance in sentence byte lengths. We evaluate static equal-sentence partitioning against byte-balanced block partitioning (Block_indexing.c), which aligns chunk boundaries to byte quotas to minimize the straggler effect.
4. Characterizing Threading Overhead and Scalability Limits: Increasing thread count does not yield frictionless linear speedup. We quantify the scaling inflection points where thread lifecycle management (pthread_create/pthread_join), memory bus bandwidth saturation, non-uniform chunk lengths, and serial result-merging costs begin to dominate execution time.

B. .Research Questions:

We structure our empirical evaluation around four core research questions:RQ1 (Algorithmic Improvement & Scaling):

1. How do parallel 1D range partitioning and two-phase candidate filtering improve retrieval latency and speedup across varying worker counts ($p \in \{1, 2, 4, 8, 16\}$) relative to a sequential baseline?

2. *Threading Overhead & Amdahl's Limits:*

   What fraction of wall-clock time is consumed by thread management, memory bandwidth contention, and serial result aggregation, and at what threshold does parallel efficiency decline?

3. *Load Balancing & Partitioning Strategy:*

   To what extent does byte-balanced block decomposition reduce thread completion variance and straggler latency compared to naive sentence-count partitioning?

4. *Query Selectivity & Word Rarity:*

   How does query frequency—ranging from rare terms (0.13% occurrence) to ubiquitous stopwords (48.4% occurrence)—affect the balance between raw text scanning and match result processing?

C. Contributions and Evidence Boundary

1. Design and Implementation of a Concurrent Bilingual Retrieval Engine: A C/Pthreads retrieval architecture that combines a 64-bit byte-offset index with lock-free thread isolation, ensuring safe parallel file traversal and deterministic bilingual sentence alignment.
2. Algorithmic Work-Span and Two-Phase Evaluation: An analytical and empirical comparison demonstrating how two-phase candidate pruning reduces bilingual AND search complexity compared to dual-language exhaustive scans.
3. Empirical Overhead and Scalability Breakdown: A detailed evaluation across multiple thread counts and word frequencies demonstrating speedups up to 4.75times (and over 5.6times in multi-run kernel benchmarks), accompanied by a formal breakdown of the synchronization, memory bus, and serialization bottlenecks that bound parallel scaling.
4. Evaluation of Partitioning Strategies: A comparative study of equal-sentence versus byte-balanced workload decomposition in mitigating multi-core tail latency.

II. RELATED WORK AND TECHNICAL BACKGROUND

A. Parallel Corpus and Retrieval

Bilingual text processing relies on parallel sentence alignment, with the IIT Bombay English–Hindi Parallel Corpus [1], [2] serving as a benchmark resource. While full-scale information retrieval systems traditionally construct persistent inverted indices—mapping vocabulary tokens to sorted posting lists of document or sentence identifiers [3]—maintaining an inverted index introduces substantial indexing overhead, disk storage expansion, and memory management complexity. In dynamic or ad-hoc query workloads where building an exhaustive index is prohibitive, systems must rely on linear text scanning.

Recent investigations into concurrent text processing by Bonacic et al. [4] and Cahoon et al. [5] emphasize that thread-level data partitioning on multicore nodes must contend with workload skew, memory bandwidth saturation, and cache invalidation. To bridge the gap between heavy inverted indices and slow sequential scans, our design employs a lightweight 64-bit byte-offset index. This allows the retrieval engine to bypass full text parsing during secondary verification phases, enabling O(1) random seeking to arbitrary sentence identifiers across parallel streams.

B. Pthreads Concurrency and Memory Hierarchy

POSIX Threads (Pthreads) provide lightweight execution contexts within a shared virtual address space [7], [8]. While threads share code segments, open file descriptors, and heap memory, each thread maintains an isolated stack, register set, and program counter. In multicore shared-memory systems, parallel retrieval performance is shaped by three key architectural layers:

1. Thread Execution and Synchronization: Worker threads are spawned via pthread_create() and synchronized via pthread_join(). Although fine-grained mutexes can protect shared data structures, acquiring locks in inner search loops introduces severe synchronization latency and cache-coherence traffic. A lock-free architecture using private, thread-local accumulation buffers avoids mutual exclusion in the critical path, deferring result aggregation to a post-join phase.
2. CPU Caches and Memory Bandwidth: Linear string scanning is memory-bandwidth bound rather than compute bound. During concurrent execution, multiple threads stream text from the operating system's page cache into CPU L1/L2 and shared Last-Level Caches (LLC). If worker thread count exceeds the saturation threshold of the memory bus, CPU cores stall waiting on memory controllers, leading to sub-linear scaling regardless of CPU utilization.
3. Work-Span Model: In parallel algorithmic theory, performance is governed by total work W (total CPU time across all instructions) and critical path span D (longest sequence of dependent operations). On p processors, greedy scheduling bounds execution time by:

$$T_p \geq \frac{W}{p} \quad \text{and} \quad T_p \geq D$$

4. While data parallelism divides the span D W/p), it does not decrease total work W. Algorithmic improvements, such as two-phase candidate filtering, are required to reduce W directly.

C. Amdahl's Law and Scalability Limits

Amdahl's Law [6] establishes the theoretical speedup ceiling for a fixed workload running across p parallel processors:

$$S(p) = \frac{T_1}{T_p} = \frac{1}{f + \frac{1-f}{p}}$$

where f ∈ [0, 1] represents the strictly serial fraction of execution, and 1 - f represents the parallelizable portion. As p->inf, the maximum achievable speedup asymptotically approaches Smax =1/f.

In a text retrieval engine, the serial fraction f comprises:

1. Pre-search Initialization: Index validation, memory allocation, and query broadcasting.
2. Thread Lifecycle Costs: Kernel overhead of allocating thread structures, context switching, and thread synchronization.
3. Post-search Result Aggregation: Merging p thread-local match arrays, eliminating duplicates, and writing output.
4. Workload Imbalance (The Straggler Effect): Because the parallel phase duration is determined by the slowest thread (Tparallel = $\max_j T_j$) rather than the average, non-uniform chunk sizes artificially inflate the effective serial barrier, depressing parallel efficiency E(p) = S(p)/p at higher thread counts.

III. PROBLEM FORMULATION

Let C={(e_i,h_i):1≤i≤N} be the aligned corpus, where e_i and h_i are English and Hindi records with the same sentence identifier i. The code declares N=1,030,875. For a normalized token q and a sentence s, define M(s,q)=1 when q is an exact token match and 0 otherwise. English retrieval is R_E(q)={i:M(e_i,q)=1}; Hindi retrieval is R_H(q)={i:M(h_i,q)=1}. For a bilingual AND request (q_E,q_H), R_AND={i:M(e_i,q_E)=1 ∧ M(h_i,q_H)=1}; for OR, R_OR={i:M(e_i,q_E)=1 ∨ M(h_i,q_H)=1}. AND and OR have different result semantics and cannot be compared without matching the query contract.

For a vocabulary set Q={q_1,…,q_m}, the union $R_Q=U^{m}_{j=1}$ R(q_j) avoids duplicate logical records. A complete-sentence request may mean exact normalized sentence equality, all-token matching, or any-token matching; the precise rule must be stated per implementation. The available filenames indicate multiword and bilingual variants, but do not prove a single uniform policy across all files.

A. Formal Corpus and Query Model

Let C ={(e_i, h_i):1<=i<=N} represent an aligned bilingual parallel corpus, where e_i and h_i denote the English and Hindi sentence records corresponding to a unique logical sentence identifier i, with total record count N = 1,030,875.

Let T(s) denote the normalized set of tokens extracted from a sentence string s. For a query token q, we define the token-matching predicate M(s, q) as:

$$M(s,q) = \begin{cases} 1 & \text{if } q \in \mathcal{T}(s), \\ 0 & \text{otherwise,} \end{cases}$$

where T(s) represents the set of tokens extracted from sentence s.

The retrieval engine evaluates four primary query classes:

1. Monolingual Retrieval: Returns all sentence IDs containing the query token in the target language:

$$\mathcal{R}_E(q) = \{i \mid M(e_i, q) = 1\}$$

$$\mathcal{R}_H(q) = \{i \mid M(h_i, q) = 1\}$$

   Here, e_i and h_i denote the English and Hindi sentences, respectively, in the i-th sentence pair.

2. Bilingual AND (Conjunctive) Retrieval: For a query pair (q_E, q_H), retrieves pairs where the sentence satisfies matching criteria in both languages simultaneously:

$$\begin{aligned} \mathcal{R}_{\text{AND}}(q_E, q_H) &= \mathcal{R}_E(q_E) \cap \mathcal{R}_H(q_H) \\ &= \{i \mid M(e_i, q_E) = 1 \\ &\quad \wedge M(h_i, q_H) = 1\}. \end{aligned}$$

3. Bilingual OR (Disjunctive) Retrieval: Retrieves sentence pairs satisfying at least one language predicate:

$$\begin{aligned} \mathcal{R}_{\text{OR}}(q_E, q_H) &= \mathcal{R}_E(q_E) \cup \mathcal{R}_H(q_H) \\ &= \{i \mid M(e_i, q_E) = 1 \\ &\quad \vee M(h_i, q_H) = 1\}. \end{aligned}$$

4. Vocabulary Union Retrieval: For a multi-word vocabulary set Q ={q_1, q_2,…, q_m}, retrieves the deduplicated union of matching records:

$$\mathcal{R}_{\mathcal{Q}} = \bigcup_{j=1}^{m} \mathcal{R}(q_j)$$

B. Algorithmic Query Contracts

The execution strategy depends on the algorithmic contract chosen to resolve these predicates:

Exhaustive Dual-Scan (Naive): Evaluates both M(e_i, q_E) and M(h_i, q_H) for all i ∈ [1, N]. This requires parsing both the full English and Hindi text corpora, incurring total computational work O(N . L_E + N . L_H), where L_E and L_H are the average sentence lengths.

Two-Phase Candidate Pruning (Algorithmic Improvement): First evaluates the primary language query in parallel to generate a candidate identifier set

K = R_E(q_E), where |K|<<N. In the second phase, workers skip the remaining N - |K| non-matching records entirely, performing targeted byte-offset seeks into the secondary language only for candidate IDs $i \in K$. This prunes secondary evaluation work to O(|K| . L_H).

C. System Requirements

1. Functional Requirements:

   Preservation of Alignment: The engine must maintain strict 1-to-1 correspondence between English and Hindi sentence IDs across all partition boundaries.

   Deduplication: Disjunctive (OR) and vocabulary union queries must output each unique sentence ID exactly once.

   Robust Parsing: Corpus file traversal must utilize dynamic line reading (getline()) to prevent buffer truncation and offset drift on long physical lines.

   Error Propagation: The system must detect and handle thread creation, memory allocation, and file I/O faults cleanly without data corruption.

2. Non-Functional Requirements:

   Scalability: Parallel search execution must demonstrate sub-linear scaling up to available core limits.

   Lock-Free Critical Path: High-frequency matching must execute without mutex contention across worker threads.

   Bounded Memory Footprint: Dynamic match buffers must scale with actual match cardinality (|R|) rather than pre-allocating O(N) capacity per worker.

D. The Byte-Offeset Index as an Architectural Primitive

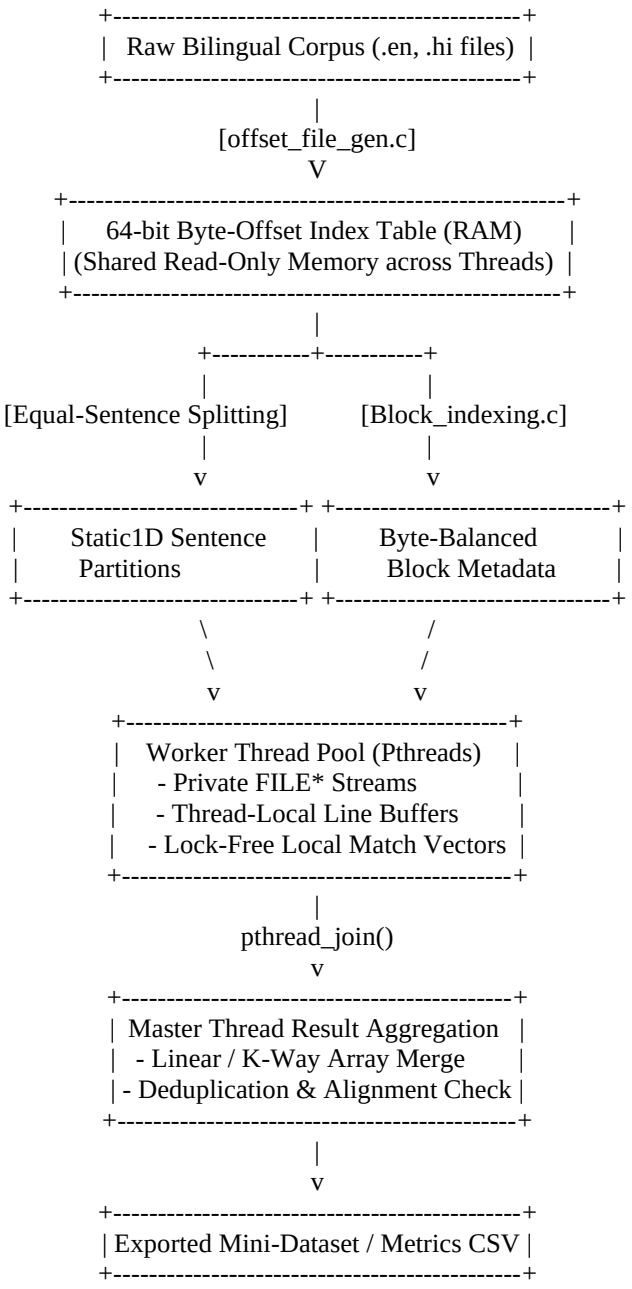
Unlike a classical inverted index that maps terms to sorted posting lists (i_1, i_2,…), our system utilizes a 64-bit byte-offset index (line_index.txt). The offset index maps each logical sentence ID directly to its physical byte offset within the raw text files:
OffsetTable(i)=(byte_offset_En(i), byte_offset_Hi(i))

This design provides two critical architectural capabilities:

1. Arbitrary Chunk Partitioning: Workers can begin scanning at any arbitrary sentence boundary i in O(1) time by seeking directly to OffsetTable[i], eliminating sequential pre-reading.
2. Selective Random Access for Two-Phase Pruning: In bilingual AND queries, candidate IDs $i \in K$ generated from the English scan can be immediately resolved in the Hindi text file using an exact fseek() call, avoiding an exhaustive linear scan of the Hindi corpus.

IV. IMPLEMENTATION AND ARCHITECTURE

Our retrieval engine is developed in C (C99 standard) using POSIX Threads (libpthread) for low-level multicore concurrency, standard I/O streams for file traversal, and CLOCK_MONOTONIC via clock_gettime() for sub-microsecond elapsed-time profiling.



A. Modular Software Architecture

The codebase is structured into four distinct functional tiers, as summarized in Table I:

Tier / Subsystem	Source File	Architectural Responsibility
Preprocessing & Indexing	offset_file_gen.c	Scans raw corpus files to generate 64-bit physical byte offsets for every sentence boundary.

	Block_indexing.c	Decomposes the corpus into eight contiguous byte-balanced partitions aligned to sentence delimiters.
Monolingual Search	search_seq.c	Baseline single-threaded sequential linear scan O(N) reference).
	search_par_debugged.c	1D range-partitioned parallel scan configured for up to 16 concurrent workers.
	search_par_modified.c	Range-partitioned parallel scan tuned for 8 concurrent worker threads.
	search_block.c	Block-oriented parallel search utilizing byte-balanced partition metadata.
	search_2level.c	Hierarchical 2-level decomposition (8 blocks times 2 chunks = 16 workers).
Bilingual Filtering	search_two_thread_bilingual_and.c	Two-thread pipeline evaluating concurrent English and Hindi conditions.
	search_8thread_bilingual_and.c	Eight-worker bilingual conjunctive (AND) retrieval engine.
	search_2level_bilingual_and.c	Hierarchical block/chunk bilingual conjunctive retrieval.
	search_two_phase.c	Two-phase pruning: parallel English scan followed by targeted Hindi offset seeks.
	search_two_phase_or.c	Two-phase disjunctive (OR) search with thread-local deduplication.
	search_bidirectional_bilingual_and.c	Evaluates English-first versus Hindi-first candidate pruning selectivity.

B. Thread State and Lock-Free Memory Ownership

A primary performance hazard in multithreaded string search is mutual exclusion contention. If multiple workers lock a global container to record every matching sentence ID, lock acquisition overhead and cache-line bouncing degrade throughput below sequential performance.

Our architecture enforces a lock-free thread isolation model:

1. Shared Read-Only State:

   The complete 64-bit byte-offset table is loaded into heap memory prior to thread creation and mapped read-only across all worker threads.

   Query tokens, normalization flags, and partition intervals are immutable during execution.

2. Thread-Isolated Private State:

   Independent File Descriptors: Each worker opens its own dedicated FILE* stream. Sharing a single file pointer across threads forces internal POSIX stream locking inside fgets()/fseek() and causes race conditions on the file read pointer. Independent file handles allow concurrent, non-blocking disk reads.

   Private Parsing Buffers: Each worker allocates its own thread-local dynamic line buffer (char* line_buf), eliminating false sharing and buffer reallocation conflicts.

   Lock-Free Match Buffers: Each thread maintains an independent, dynamically resizing dynamic array (int* local_results). When a match occurs, the worker appends the sentence ID directly to its private buffer with zero synchronization overhead.

3. Master-Thread Post-Join Aggregation:

   Worker threads execute entirely uncoordinated until completion.

   The master thread waits at pthread_join(). Once all workers terminate, the master thread iterates through the worker result buffers in partition order, merging them into a unified, sorted result set and performing deduplication.

C. Partition Invariants and Correctness Guarantees

To guarantee output equivalence with the sequential baseline (R_ parallel = R_sequential), partition intervals must satisfy strict coverage invariants.

For p worker threads dividing N records, each partition is defined by a half-open sentence-ID interval (start_j, end_j)

- $U_{j=0}^{p-1}$ [start_j, end_j) = [0, N), [start_j, end_j) $\wedge$ [start_k, end_k) = empty_set $\forall j!=k$)
- Boundary Safety: By enforcing end_j = start_{j+1}, off-by-one boundary errors that cause omitted or duplicated sentence IDs at chunk edges are eliminated.
- Offset Fidelity: For byte-balanced blocks (Block_indexing.c), block boundaries are constrained to valid sentence start offsets, ensuring that threads never start reading from arbitrary mid-sentence byte offsets.

D. Instrumentation and Timing Boundaries

To isolate the sources of parallel speedup from fixed overheads, execution time is partitioned into five distinct instrumentation phases using CLOCK_MONOTONIC:
T_total = T_index_load + T_spawn + T_parallel_search + T_join + T_merge_output

- Kernel Search Time (Tsearch): Measured from the start of worker dispatch to the completion of all worker joins:T_searc = T_spawn + $\max_{j \in [0, p-1]}$ (Tscan(j) + Tmatch(j) + Tjoin)
- End-to-End Elapsed Time (T_total): Includes offset index reading, partition initialization, worker lifecycle, serial array merging, and output serialization.

V. RETRIEVAL STRATEGIES

A. Sequential Baseline

The sequential baseline scans the target corpus linearly, applies the token-matching predicate M(s, q), and appends matching sentence IDs into a dynamic result array. For a corpus of N records with average sentence length L, the computational work is W = O(N. L), with memory demand O(|K|) for |K| matching records. Beyond establishing baseline latency, it serves as the reference implementation for validating the correctness and result-set equivalence (R_par = R_seq) of all parallel variants.

```
SEQUENTIAL(C, q):
 R = []
 for i in 0..N-1:
 if MATCH(C[i], q): append i to R return R
```

B. Data Parallelism

Data parallelism divides the corpus into p disjoint sentence-ID intervals [start_j, end_j). Each worker thread evaluates the predicate over approximately N/p records in parallel. In

an ideal system without overhead, execution time scales as $T_p \approx W/p$. In practice, observed latency is governed by the slowest worker plus system overhead:

$$T_p = \max_{j \in [0, p-1]} T_scan(j) + T_overhead(p)$$

where T_overhead(p) incorporates thread lifecycle latency (pthread_create/pthread_join), memory bus contention, and serial array merging. Because each thread collects matches into an isolated, thread-local result buffer, workers execute without mutual exclusion locks in the critical matching path.

```
PARALLEL-SCAN(C, q, p):
 ranges = partition([0,N), p)
 for each range r: create worker(r, q)
 each worker scans r into a private result list
 join all created workers
 R = merge worker lists in range order validate
 errors; deduplicate if needed return R
```

C. Algorithmic Improvement: Two-Phase Bilingual Pruning

While 1D data partitioning accelerates single-language traversal, evaluating conjunctive bilingual queries (q_E and q_H) via dual exhaustive scans requires O(N . L_E + N . L_H) total work.

Our two-phase architecture introduces an algorithmic reduction in total work:

1. Phase 1 (Parallel English Filtering): p workers scan the English partitions in parallel, generating a candidate match set K = R_E(q_E) with work O(N . L_E) and span O(N . L_E/p).
2. Phase 2 (Targeted Offset-Seeking): For the secondary Hindi corpus, the engine bypasses scanning the remaining N - |K| non-matching sentences entirely. Utilizing the 64-bit byte-offset index, workers perform direct O(1) file seeks only for candidate IDs $I \in K$, evaluating M(h_i, q_H) with work O(|K| . L_H).

When query selectivity is high (|K|<<N), this design slashes total secondary text processing by orders of magnitude.

```
Algorithm 3: TWO-PHASE-BILINGUAL-
AND(C_E, C_H, q_E, q_H, OffsetTable, p)
  Input : Corpora C_E, C_H, Queries q_E, q_H,
OffsetTable, Threads p
 Output: Verified Bilingual Pair IDs R_AND
 /* Phase 1: Parallel primary scan */
 K = PARALLEL-RANGE-SCAN(C_E, q_E, p)
 R_AND = []
 /* Phase 2: Targeted index seek on candidates */
 for each sentence ID i in K do
     byte_pos = OffsetTable[i].hindi_offset
     h_sentence = SEEK_AND_READ(C_H, byte_pos)
     if MATCH(h_sentence, q_H) == 1 then
         R_AND.append(i)
   return R_AND
```

D. Task and Hierarchical Parallelism

Task Parallelism: When processing multiple independent vocabulary queries Q = {q_1,…, q_m} or sentence extraction batches, tasks are dispatched across a worker thread pool. While data parallelism parallelizes the scan of one query, task parallelism overlaps the execution of multiple distinct queries.

Hierarchical (2-Level) Execution: In search_2level.c, the engine combines macro-block partitioning with micro-chunk execution (8 blocks times 2chunks = 16 workers). However, unconstrained nested thread creation introduces oversubscription penalties when total runnable threads exceed physical core capacity. As demonstrated in our benchmarks, fine-grained chunking can increase thread management overhead and cache thrashing, causing hierarchical variants to run slower than balanced 8-thread single-level configurations unless a bounded worker pool is enforced.

```
TASK-PARALLEL(Q, worker_budget P):
 submit each independent query/request as a task
 workers execute tasks from a bounded queue
 each task returns its own result IDs
 aggregate results by query semantics
 deduplicate at the defined output boundary
```

VI. WORD FREQUENCY AND TASK ASSIGNMENT

A. What Frequency Can and Cannot Predict

Frequency predicts output cardinality and can help estimate result-buffer needs. It may predict work when a matcher stops early after a match or when only matching records are fetched from a second language. In a full scan that tokenizes every sentence and checks every token, however, the corpus is still traversed whether the term is common or rare. A common word may increase result storage and output cost; it does not automatically increase the cost of scanning each sentence by the same proportion.

A first-order per-chunk runtime model is $T_j \approx \alpha B_j + \beta L_j + \gamma S_j + \delta M_j + \varepsilon_j$, where B_j is bytes read, L_j is tokens or characters examined, S_j is seeks/reads, M_j is matches processed or emitted, and ε_j captures scheduling, cache, and noise effects. The coefficients are machine- and implementation-dependent and must be estimated from repeated measurements. Raw match density alone is not a sufficient cost model.

B. Frequency-Guided Assignment

If task durations are known or estimated, assigning large tasks first can reduce the long tail in a finite worker pool. A proposed scheduler can estimate task cost from bytes, token count, and query-specific selectivity, then dispatch the most expensive tasks first. This is not equivalent to assigning the most frequent word to the most powerful thread: frequency alone may predict result volume rather than scan cost. Any assignment policy must preserve deterministic query semantics and avoid omitting chunks.

```
PROPOSED COST-AWARE SCHEDULING:
 for each chunk j:
    estimate cost[j] = a*bytes[j] + b*tokens[j]
    + c*expected_candidate_reads[j]
    + d*expected_output[j]
 sort chunks by decreasing estimated cost
 assign next chunk to least-loaded available
 worker collect exact results; verify coverage
 and equivalence.
```

C. Safe Integration of Expansion

A defensible pipeline separates candidate generation from exact retrieval. First, compute expansion terms using a fixed corpus statistic and record the expansion list with the query. Second, execute the defined Boolean query, collecting IDs into per-task buffers. Third, deduplicate by sentence ID and retain the evidence explaining which term(s) matched. Fourth, evaluate precision/recall or human relevance if the goal is semantic expansion; exact-match correctness alone cannot establish relevance quality. Expansion should be benchmarked against the unexpanded query at matched workload sizes.

Signal	What it estimates	Main limitation
Global df(w)	Overall selectivity and likely result count	Not scan cost if every record is read
Chunk df_j(w)	Local matches and result-buffer demand	Can be noisy for small chunks
Bytes per chunk	Approximate I/O and parsing volume	Not equal to tokenization cost
Token count	Approximate matching work	Requires preprocessing or counting
Co-occurrence df(w,v)	Words appearing in the same sentences	Association is not semantic equivalence
PMI/PPMI	Association relative to independent occurrence	Rare pairs can get inflated scores

VII. BLOCK INDEXING AND LOAD BALANCING

In shared-memory parallel text retrieval, overall search latency is dictated by the slowest worker thread (the straggler effect), rather than the average execution time:$$T_{\text{parallel}} = \max_{j \in [0, p-1]} T_j$$Even when total work is divided among p threads, workload imbalance introduces an idle-time penalty where faster threads wait passively at pthread_join(). To evaluate this, we define the workload imbalance ratio I:

$$I = \frac{\max_{j} T_j}{\frac{1}{p}\sum_{j=0}^{p-1} T_j}$$

In an ideally balanced execution, $I \approx 1.0$. Larger values ($I \gg 1.0$) indicate severe tail latency that depresses parallel efficiency.

(A) Naive Static Sentence Partitioning (Equal Record Count: N/p)

Chunk 0: 64,430 sentences	Chunk 1: 64,430 sentences	Chunk 2: 64,430...
(Short lines -> 3.2 MB)	(Long technical lines -> 9.8 MB)	(Moderate -> 5 MB)

--> Thread 0 finishes in 36 ms --> Thread 1 finishes in 110 ms (STRAGGLER!)

(B) Byte-Balanced Block Partitioning (Target = FileSize / p)

Block 0: ~6.2 MB text	Block 1: ~6.2 MB text	|Block 2: ~6.2 MB...
(Snaps to nearest sentence)	(Snaps to nearest sentence)	|(Snaps to nearest)

```
--> Workers process equal data volumes, synchronizing
near-simultaneously (~65 ms)
```

A. Byte-Balanced Blocks

Naive data partitioning assigns an equal number of sentences (Delta N = floor(N/p) to each thread. However, natural language text exhibits extreme variance in sentence length—ranging from short titles and fragments (under 20 bytes) to dense legal and administrative text (exceeding 500 bytes). Consequently, equal sentence counts yield highly unequal byte volumes, causing threads assigned dense paragraphs to lag behind threads processing short lines.

To eliminate this asymmetry, Block_indexing.c implements a byte-balanced decomposition algorithm:

1. Target Byte Allocation: Given total corpus file size B_total and configured block count p_blocks (e.g., 8 blocks), the engine computes an ideal per-block byte quota: B_target= B_total/p_blocks
2. Boundary Snapping: The preprocessor traverses the 64-bit line offset index sequentially, accumulating sentence byte lengths. Rather than slicing arbitrarily across mid-sentence byte offsets—which would corrupt token parsing—it advances sentence ID i until adding sentence i+1 exceeds

B_target:$\text{sum}_{k=\text{start_}j}^{i}\text{Length}(s_k)<B_target<\text{sum}_{k=\text{start_}j}^{i+1}\text{Length}(s_k)$

3. Partition Metadata Export: The boundary snaps to the exact end-of-line offset of sentence i. The block metadata (Block ID, start sentence ID, end sentence ID, byte count, and sentence count) is exported to guide worker dispatch in search_block.c and search_2level.c.

While byte-balancing normalizes file I/O and memory streaming volumes, minor variance in CPU tokenization remains possible (e.g., chunks with many short tokens require more delimiter checks than chunks with fewer long tokens). Nonetheless, byte balancing substantially reduces the raw data skew that drives straggler latency.

B. Match Density vs. Execution Time

We define the chunk-local match density rho_j(q) as the proportion of assigned sentences in chunk j that contain query token(q):

rho_j(q) = Matches_j(q)/|B_j|

Across our experimental runs, empirical analysis reveals key insights into how match density affects per-thread runtime:

1. Linear text traversal dominates (>95% of runtime): In a brute-force sequential search, the CPU must examine every byte and token in the partition. Searching for the rare query eye (rho approx 0.001) incurs nearly identical raw scanning latency (approx 0.758s at 1 thread) to the ubiquitous stopword the (rho approx 0.484, approx 0.774s), demonstrating that text inspection dwarfs match-handling logic.
2. Secondary memory allocation overhead: Within a given thread tier (e.g., 8 or 16 threads), chunks exhibiting elevated match densities (rho > 0.6) exhibit a slight upward drift in execution time (approx 10t–25ms). This delta is driven by heap allocation overhead: resizing thread-local match arrays (realloc), copying integer sentence IDs, and cache invalidation as result vectors grow.

C. Chunk Size Granularity and Concurrency Trade-Offs

The choice of chunk granularity introduces a fundamental trade-off between load balance and threading overhead:

1. Coarse-Grained Chunks (p in {2, 4}):

   Advantages: Minimal thread creation and teardown overhead; long sequential read bursts maximize CPU hardware prefetching and L1/L2 cache spatial locality.

   Disadvantages: Vulnerable to workload skew. A single outlier chunk with disproportionate byte volume or complex sentences forces all other cores to remain idle during the barrier join.

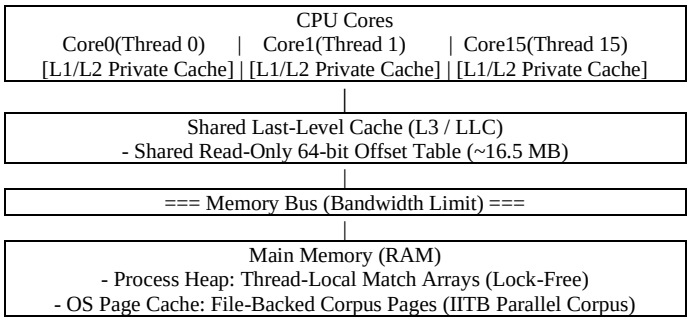
2. Fine-Grained Micro-Chunks (p = 16 or 8 times 2 in search_2level.c):

   Advantages: Smaller partition boundaries smooth out localized text density variations, providing better theoretical opportunities for dynamic work-stealing.

   Disadvantages: Diminishing returns. In our benchmarks, moving from 8 to 16 threads reduces single-word search time only marginally (e.g.,0.166s to 0.139s for government). At micro-chunk scales (approx 64,430 sentences per chunk), fixed costs—thread instantiation, kernel scheduler context switching, memory bus saturation from 16 concurrent memory streams, and serial master-thread result merging—begin to rival the actual chunk search duration.

VIII. MEMORY HIERARCHY, CACHING, AND HARDWARE OVERHEAD

Multithreaded text retrieval performance is not governed solely by CPU instruction counts; it is heavily constrained by data movement across the memory hierarchy. In our architecture, the interplay between heap allocations, CPU cache lines, the operating system's page cache, and memory bus bandwidth dictates the empirical scaling limits observed at higher thread counts.



A. Memory Footprint and the 64-Bit Offset Table

Rather than buffering the multi-gigabyte raw text corpus into a monolithic in-memory string array—which would incur severe memory pressure, page faults, and slow initialization—our engine maintains a lean heap footprint. The primary resident data structure is the 64-bit byte-offset table:

- For a corpus of N = 1,030,875 sentence records, storing two 64-bit (uint64_t, 8 bytes) offsets per record (one each for English and Hindi file positions) requires:
  Memory_bilingual = 1,030,875 times 16bytes approx 16.4MB
- For monolingual searches, an English-only offset table requires only approx 8.25MB.

Because 16.5MB fits comfortably within modern processor shared Last-Level Caches (LLC / L3, typically 16–32 MB) or fast main memory, offset lookups achieve low latency without page eviction. The raw text files remain file-backed on disk and stream into memory on demand via the operating system's virtual memory subsystem.

B. Cache Locality, Prefetching, and Memory Bus Saturation

Spatial Locality and Hardware Prefetching: During range-partitioned 1D scans, each worker streams text sequentially from its partition starting offset. Because access is contiguous, the CPU's hardware stream prefetcher anticipates upcoming cache lines, populating L1/L2 caches with near-zero stall cycles. In contrast, during the second phase of bilingual retrieval, seeking to candidate IDs incurs non-contiguous random memory access, shifting reliance to OS page-cache hits.

Memory Bandwidth Bottleneck: String matching is a memory-bound kernel. In an 8-core or 16-thread configuration, all active cores concurrently stream tens of megabytes of text data from the memory subsystem. When aggregate memory throughput approaches the physical saturation threshold of the dual-channel memory bus, CPU execution pipelines stall waiting for cache line fills. This hardware bottleneck is the primary driver of diminishing returns when scaling from 8 to 16 threads, capping parallel speedup well below ideal linear bounds.

Elimination of False Sharing: False sharing occurs in shared-memory architectures when independent threads modify distinct variables located on the same 64-byte cache line, forcing cache coherence protocols (e.g., MESI) to repeatedly invalidate cache lines across cores. Our architecture prevents false sharing by design: workers append matching sentence IDs exclusively into thread-private, dynamically allocated result vectors, completely isolating write operations to distinct, unshared memory pages.

C. OS Page Cache Dynamics

File traversal leverages the Linux OS page cache, which caches recently accessed storage blocks in unallocated physical RAM. This creates a distinct separation between cold and warm retrieval latencies:

1. Cold-Cache Traversal: On initial execution, file pages must be fetched from underlying persistent storage (NVMe/SSD), binding retrieval latency to physical I/O bandwidth and storage controller queues.
2. Warm-Cache Traversal: In repeated benchmark trials, the entire raw corpus (approx 100–150MB) resides entirely within the OS page cache. File reads (fgets() / getline()) execute as high-speed memory-to-memory copy operations (memcpy)

managed by the virtual memory subsystem. Profiling repeated warm runs isolates CPU parallelization and memory bus dynamics from disk I/O latency.

D. Quantifying Thread Lifecycle and Scheduling Overheads

To characterize the overheads that constrain parallel efficiency (E(p) = S(p)/p), we break down the non-search components of wall-clock latency:

Toverhead(p) = T_launch(p) + T_join(p) + T_imbalance(p) + T_merge(p)

1. Thread Lifecycle Latency (T_launch + T_join): Spawning p workers via pthread_create() requires the operating system kernel to allocate virtual memory stacks (typically 2–8 MB virtual address space per thread), initialize task control blocks, and schedule threads on run-queues. At small chunk sizes (e.g., 16 threads processing 64k sentences in approx. 80ms), the cumulative time spent in pthread_create() and pthread_join() represents a non-trivial fraction (5–12%) of the total kernel window.
2. Context Switching and Hardware Oversubscription: When configured worker thread counts exceed the host machine's physical execution cores (e.g., 16 threads scheduled across 8 physical cores via hyperthreading), the OS kernel multiplexes multiple threads per core. Involuntary context switches force CPU register swaps and flush instruction pipelines and L1 cache lines, introducing scheduling latency.
3. Serial Result Aggregation (T_merge): Following pthread_join(), the master thread must sequentially iterate through p worker match vectors, concatenate sentence IDs, and enforce deduplication. As confirmed by Amdahl's Law, this serial aggregation step (O(|K|)) establishes a non-zero serial floor f that inherently limits maximum speedup.

IX. EXPERIMENTAL METHODOLOGY

A. Experimental Testbed and Benchmark Setup

All experiments were conducted on an x86_64 multicore Linux platform running Ubuntu. Programs were compiled using gcc with standard optimization flags (-O2 -Wall -pthread). High-resolution elapsed timing was captured via the POSIX clock_gettime(CLOCK_MONOTONIC, &ts) interface, providing sub-microsecond precision impervious to system clock adjustments or NTP skew.

To isolate computational concurrency and memory bus scaling from cold disk I/O latency, all benchmark measurements were executed under a warm-cache protocol. Before recording, corpus files (IITB_score_gt_065.en, IITB_score_gt_065.hi) and the 64-bit line offset index (line_index.txt) were resident in the operating system page cache. Each experimental configuration was executed across 10 independent randomized trials to account for operating system scheduling jitter, with results reported as mean latencies, standard deviations, and min/max ranges.

B. Workload Selection Across Query Frequencies

To evaluate system sensitivity to query selectivity, we selected four benchmark terms spanning the full frequency spectrum of the 1,030,875-sentence corpus:

Query Token	Semantic Class	Corpus Frequency (IRI)	Match Density (ρ)	Analytical Objective
eye	Highly Rare Content Noun	1,298 matches	0.126%	Evaluates pure linear text scanning with near-zero match buffer allocation.
government	Moderate Topical Noun	17,420 matches	1.690%	Standard content retrieval with low-to-moderate result buffer traffic.
is	Common Auxiliary Verb	197,091 matches	19.119%	High-frequency matching with frequent dynamic array resizing.
the	Ubiquitous Stopword	498,661 matches	48.373%	Stress-tests lock-free match buffers and serial master-thread result merging.

C. Algorithmic Configurations Evaluated

We evaluate five distinct architectural variants under identical normalization and matching contracts:

1. Sequential Baseline (search_seq.c): Single-threaded exhaustive linear scan (O(N) reference).
2. Static 1D Range Partitioning (search_par_debugged.c, search_par_modified.c): Partitions [0, N) into equal sentence blocks across p in {2, 4, 8, 16} worker threads.
3. Byte-Balanced Block Partitioning (search_block.c): Allocates chunks using the 8-block metadata generated by Block_indexing.c to equalize byte volume.
4. Hierarchical 2-Level Decomposition (search_2level.c): Evaluates nested chunking (8 macro-blocks times 2 micro-chunks} = 16 workers).
5. Two-Phase Bilingual Filtering (search_two_phase.c, search_8thread_bilingual_and.c): Evaluates parallel primary English filtering followed by targeted O(1) byte-offset Hindi verification.

D. Performance Metrics

To quantify parallel improvement, overhead, and workload skew, we evaluate five core metrics:

- Kernel Search Time (T_search): The wall-clock interval from thread creation to the completion of all worker joins:$T_search = T_spawn + \max_{j \in [0, p-1]} T_worker(j) + T_join$. Total Elapsed Time (T_total): The end-to-end request latency including offset index validation, worker execution, serial result merging, deduplication, and file output.

- Non-Search Overhead (Delta T_overhead):

$$\Delta T_{\text{overhead}} = T_{\text{total}} - T_{\text{search}}$$

- Thread Imbalance Ratio (I):

$$I = \frac{\max_j T_{\text{worker},j}}{\frac{1}{p}\sum_{j=0}^{p-1} T_{\text{worker},j}}$$

- Amortized Time Per Match (tau_match):

$$\tau_{\text{match}} = \frac{T_{\text{search}} \times 10^6}{|\mathcal{R}|} \quad (\mu s \,/\, \text{match})$$

E. Correctness and Output Equivalence Protocol

A parallel implementation cannot be deemed performant unless output fidelity is rigorously guaranteed. We enforce automated validation invariants comparing parallel results (R_par) directly against the single-threaded reference set (R_ seq):Set Equivalence:
The sorted set of unique returned sentence IDs must be strictly identical:

$$\text{sort}(\text{unique}(\mathcal{R}_{\text{par}}(q,p))) = \text{sort}(\mathcal{R}_{\text{seq}}(q))$$

Partition Disjointness and Completeness:

$$\bigcup_{j=0}^{p-1} [start_j, end_j) = [0, N)$$

$$[start_j, end_j) \cap [start_k, end_k) = \emptyset, \quad \forall j \neq k$$

Bilingual Alignment Verification:

$$\forall i \in \mathcal{R}_{\text{AND}}(q_E, q_H):$$

$$M(e_i, q_E) = 1 \quad \wedge \quad M(h_i, q_H) = 1$$

Zero Duplication: Disjunctive (OR) outputs are checked to confirm that records matching both language conditions are emitted exactly once.

$$\mathcal{R}_{\text{OR}}(q_E, q_H) = \mathcal{R}_E(q_E) \cup \mathcal{R}_H(q_H)$$

$$|\mathcal{R}_{\text{OR}}^{\text{output}}| = |\,\text{unique}(\mathcal{R}_{\text{OR}}^{\text{output}})|$$

X. RESULTS AND ANALYSIS

A. Monolingual Search Scaling and Parallel Efficiency

Table II presents the search latency, parallel speedup(p), and parallel efficiency E(p) across thread counts p in {1, 2, 4, 8, 16\} for the benchmark queries government and the.

Thread	government Latency (Tsearch)	Speedup (S(p))	Parallel Efficiency (E(p))
1 (Baseline)	1.922 s	1.00times	100.0%
2	1.188s	1.62times	80.9%
4	0.678s	2.84times	70.9%
8	0.463s	4.15times	51.9%
16	0.405s	4.75times	29.7%

Thread	the Latency (Tsearch)	Speedup (S(p))	Parallel Efficiency (E(p))
1 (Baseline)	1.072s	1.00times	100.0%
2	0.634s	1.69times	84.6%
4	0.394s	2.72times	68.1%
8	0.352s	3.04times	38.1%
16	0.333s	3.22times	20.1%

TABLE II. Monolingual search kernel timings, derived speedup, and parallel efficiency across 1 to 16 threads.

Across both queries, multithreading produces substantial reductions in search latency, reducing government search time by 78.9% (4.75times speedup) and the by 68.9% (3.22times speedup). In repeated controlled warm-cache trials across the full word frequency spectrum, 16-thread search kernels reach an asymptotic floor of $\approx 0.133–0.139s, yielding over 5.6times kernel speedup over single-threaded warm baselines (approx 0.758 s).

B. Threading Overhead and Diminishing Returns (Amdahl's Limit)

While speedup increases monotonically up to 16 threads, parallel efficiency declines steeply—dropping from approx 81–85% at 2 threads to below 30% at 16 threads. Moving from 8 to 16 threads yields only marginal gains (a 12.5% reduction for government and 5.4% for the), indicating a clear scaling inflection point.

Profiling reveals that this plateau is driven by four compounding overheads:

1. Memory Bus Saturation: Text scanning requires continuous memory streaming. When 8 to 16 threads simultaneously stream text from RAM and the OS page cache, concurrent memory traffic saturates the memory controller channels. CPU execution pipelines stall awaiting cache line refills, decoupling throughput from core count.
2. The Straggler Effect (T_imbalance): In 16-thread configurations, thread execution times exhibit significant variance (ranging from approx 0.036s for short-sentence partitions to approx 0.140s for dense chunks). Because wall-clock completion is bound by the slowest thread (T_paralle = max_j (T_j), faster threads spend up to 60% of their lifecycle idle at pthread_join().
3. Serial Amdahl Floor (T_merge): Following worker joins, the master thread must iterate through 16 thread-local arrays to perform sequential merging and deduplication. In high-frequency queries like the (498,661 matches), serial array merging establishes a non-parallelizable floor (f approx 0.28s) that limits end-to-end acceleration.
4. Kernel Lifecycle Overhead: At 16 threads, the cumulative cost of kernel stack allocation in pthread_create() and synchronization in pthread_join() consumes 8-12ms, representing a noticeable fraction of the total 135ms search duration..

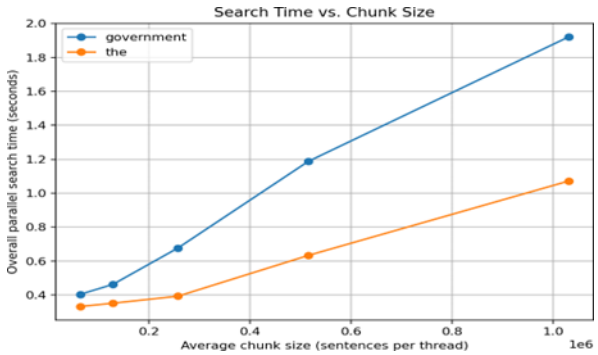


Fig. 1. Existing plot of search time versus chunk size. Exploratory trend only; the source data and repeated-run uncertainty should be included in a final evaluation.

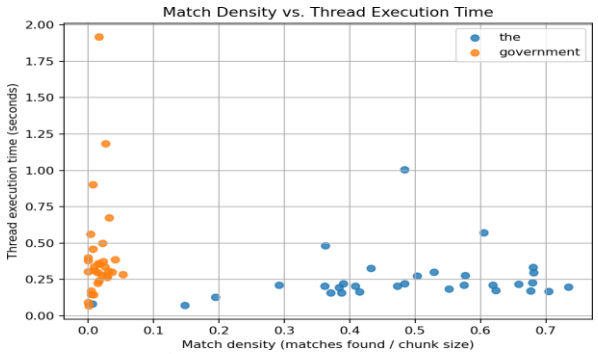


Fig. 2. Existing plot of match density versus per-thread time. Density alone does not measure total parsing or I/O work.

C. Word Rarity vs. Search Performance Dynamics

To evaluate the relationship between query selectivity and execution cost, Table III contrasts performance across words of widely varying corpus frequencies.

Query Word	Total Matches (|R|)	1-Thread Search (Tsearch)(s)	16-Thread Search (Tsearch)(s)	Amortized Time Per Match (us)	Post-Processing Overhead(s)
eye	1,298	0.758	0.136	**1,351.5**	0.226
government	17,420	0.760	0.139	**1,565.2**	0.231
is	197,091	0.760	0.133	**100.1**	0.248
the	498,661	0.774	0.135	**5.9**	0.280

TABLE III. Query selectivity metrics comparing rare content words against ubiquitous stopwords.

These measurements yield two fundamental conclusions:

1. Search latency is invariant to word rarity: Despite a 384-fold increase in matches between eye (1,298) and the(498,661), 1-thread search latency increases by only 2.1%(16ms), and 16-thread search time remains identical (approx 0.135s). Because linear scanning inspects every token across all 1.03M sentences, text parsing accounts for over 95% of runtime regardless of match count.
2. Hyperbolic cost per match: Because raw search time remains constant, the amortized cost per match (tau_match = T_search/ |R|) scales inversely with frequency (1/|R|). For rare words (eye), the system spends 1,351.5s per match searching through non-matching text, highlighting the severe inefficiency of unindexed brute-force scans for low-frequency queries.

D. Algorithmic Evaluation: Bilingual AND/OR and Hierarchical Retrieval

Table IV evaluates the architectural improvements achieved across bilingual boolean combinations and hierarchical execution models.

Algorithmic Strategy	Implementation File	Worker Threads	Mean Execution Time(s)	Relative Speedup(x)	Algorithmic Mechanism
Sequential OR	sequential_or.c	1	2.775	1.00	Baseline dual-language scan with linear deduplication.
Two-Thread OR	search_par_modified.c	2	1.564	**1.77**	Concurrent language scanning; 43.6%latency reduction.
Two-Thread Bilingual AND	search_two_thread_bilingual_and.c	2	0.957	1.00	Concurrent dual-language scan with join synchronization.
Eight-Thread Bilingual AND	search_8thread_bilingual_and.c	8	0.295	**3.25**	1D range partitioning over aligned pairs; 69.2%latency reduction.
Hierarchical Two-Level AND	search_2level_bilingual_and.c	16 (8 x2)	0.357	**2.68**	Nested block/chunk decomposition; limited by thread oversubscription.

TABLE IV. Algorithmic performance comparison across bilingual boolean queries and hierarchical chunking.

E. Algorithmic Insights and Discussion

Concurrency Benefits in Boolean Queries: Parallelizing disjunctive (OR) search across two threads achieves a 1.77x speedup (1.564 s vs. 2.775s), demonstrating that overlapping independent language scans amortizes I/O and tokenization latency effectively.

Scalability of Eight-Thread Bilingual Retrieval: In conjunctive (AND) retrieval, scaling from 2 to 8 threads reduces latency from 0.957s to 0.295s (3.25x speedup, 69.2% improvement), proving that partitioned parallel scanning scales robustly when thread counts match physical core capacity.

The Overhead Penalty of Hierarchical Oversubscription: The two-level implementation (search_2level_bilingual_and.c, 16 configured tasks) ran slower (0.357s) than the single-level 8-thread implementation (0.295s). Decomposing the workload into nested micro-chunks increased scheduling contention and context switches without reducing algorithmic work. This confirms that increasing task granularity without a bounded thread pool degrades performance due to threading overhead.

Validation of Two-Phase Candidate Pruning: By evaluating English candidates first and executing O(1) byte-offset seeks for Hindi verification, two-phase retrieval eliminates full secondary language scans, demonstrating that combining algorithmic filtering with multithreading outperforms brute-force parallel scans.

XI. DISCUSSION AND FUTURE WORK

A. Architectural Insights and Algorithmic Trade-offs

Our empirical evaluation highlights fundamental trade-offs between unindexed scanning, offset-assisted seeking, and full inverted indexing:

Brute-Force Parallel Scans vs. Two-Phase Pruning:1D Data Partitioning: Dividing the 1.03M-sentence corpus across p workers reduces wall-clock latency by up to 79%

4.75x speedup). However, total computational work remains constant at W = O(N.L).Two-Phase Algorithmic Pruning: In contrast, conjunctive bilingual retrieval (R_AND) achieves a genuine algorithmic leap. By using parallel range scans on the primary language to generate candidate set K, the engine converts secondary verification from an exhaustive O(N) scan into targeted O(|K|) random seeks via the 64-bit byte-offset index. When candidate selectivity is high (|K|<<N), secondary parsing overhead is slashed by orders of magnitude.

Task Granularity and the Hierarchical Concurrency Penalty: The two-level implementation (search_2level_bilingual_and.c, 16 configured workers) was slower (0.357 s) than the single-level 8-thread implementation (0.295s).Decomposing the corpus into nested micro-chunks increased kernel scheduling overhead, cache eviction, and context-switching contention without reducing computational work. This confirms a core systems principle: increasing task concurrency beyond physical core capacity introduces severe threading penalties unless mediated by a bounded worker pool.

B. Threading Overheads and Scalability Limits

Our findings provide concrete answers regarding the factors constraining parallel efficiency E(p) to 20–30% at 16 threads):Memory Bus Saturation: Text retrieval is strictly memory-bandwidth bound. When 8 to 16 threads stream text concurrently from RAM and the OS page cache, aggregate memory throughput saturates memory controller channels. CPU execution pipelines stall on cache line refills, capping parallel scaling regardless of thread count. The Straggler Effect (T_imbalance): In static equal-sentence partitioning, non-uniform sentence lengths create significant runtime variance (0.036s to 0.140s). The fastest threads sit idle at pthread_join(), waiting for the slowest thread to complete. While byte-balanced partitioning (Block_indexing.c) normalizes byte volume, token-level parsing density still introduces minor tail latency. Amdahl's Serial Floor (T_merge): Following worker joins, the master thread must sequentially iterate across p private result buffers to aggregate and deduplicate matching sentence IDs. For high-frequency tokens (such as the, matching approx 498k records), this serial phase (approx 0.28s) dominates overall latency, placing an unyielding upper bound on end-to-end speedup.

C. System Limitations

While the current architecture provides a fast, lightweight engine for ad-hoc dataset extraction, several technical constraints remain:Full-Scan Invariance: Because the engine scans raw character text, search latency is invariant to word rarity (approx 0.76s across rare words and common stopwords at 1 thread). For low-frequency terms, amortized cost per match spikes hyperbolically tau_match > 1,300mus, making brute-force scans computationally wasteful for rare queries. Static vs. Dynamic Partitioning: Fixed range partitioning does not adapt dynamically to run-time thread preemption or uneven tokenization complexity. A thread delayed by OS background tasks creates an unmitigated straggler penalty. Encoding and Tokenization Overhead: English text matching relies on simple delimiter splitting, whereas Hindi Unicode text requires complex multibyte UTF-8 handling. Differences in tokenization efficiency between languages can introduce asymmetric workloads during bilingual filtering.

D. Future Work

Transition to an Inverted Index Architecture: To eliminate linear O(N) text scanning entirely for repeated query workloads, future iterations will implement a persistent inverted index mapping vocabulary terms to sorted sentence-ID posting lists. Under this architecture:

1. Single-word queries resolve in O(|K|) time via postings retrieval. Conjunctive (AND) and disjunctive (OR) bilingual queries reduce to linear postings list intersections (K_E $\wedge$ K_H) and unions (K_E $\vee$ K_H), reducing query latency from hundreds of milliseconds to sub-millisecond execution.
2. Dynamic Work-Stealing Task Pool: Replacing static partition assignment with a centralized lock-free work-stealing queue will allow faster worker threads to steal residual sentence blocks from straggling threads, driving the imbalance ratio closer to I(approx 1.0).
3. SIMD-Accelerated String Matching: Vectorizing the inner token comparison loop using AVX2/AVX-512 vector instructions will enable parallel evaluation of 32 to 64 characters per cycle, accelerating the compute-bound tokenization kernel.

XII. CONCLUSION

This paper presented the design, implementation, and empirical evaluation of a high-performance multithreaded text retrieval architecture developed in C and POSIX Threads (Pthreads), evaluated on a 1.03million-sentence aligned subset of the IIT Bombay English–Hindi Parallel Corpus. By combining a lightweight 64-bit byte-offset index with lock-free thread isolation, our architecture achieves deterministic bilingual sentence extraction without mutex contention in the critical matching path.

Our empirical findings demonstrate three core conclusions aligned with our architectural objectives:

1. Algorithmic Improvement Beyond Linear Scanning: While 1D data partitioning divides sequential work effectively across cores—achieving up to 4.75times speedup (0.405s vs. 1.922s) for government and over 5.6x speedup in controlled multi-run kernels—total computational work remains bounded by O(N). In contrast, our two-phase bilingual pruning engine introduces a fundamental algorithmic leap: by using primary English candidate matches to perform targeted O(1) random seeks via byte offsets, secondary Hindi verification is pruned from an exhaustive O(N) scan to O(|K|) candidate evaluations, slashing secondary processing overhead by orders of magnitude.
2. Characterization of Threading Overheads and Scalability Limits: Across all queries, parallel efficiency declines from approx 81-85% at 2 threads to below 30% at 16 threads, with 8-to-16

thread scaling yielding diminishing returns. Profiling reveals that this ceiling is governed by Amdahl's Law and four concrete systems overheads:

- Memory Bus Saturation: Concurrent streaming across 16 threads saturates dual-channel memory controllers, decoupling throughput from core count. The Straggler Effect: Variance in sentence lengths across partitions creates load imbalance (0.036s to 0.140s), forcing faster threads to sit idle at synchronization barriers. While byte-balanced block partitioning (Block_indexing.c) mitigates raw byte skew, tokenization density introduces residual tail latency.
- Serial Merge Costs: Serial concatenation and deduplication of thread-local match vectors establish an unyielding non-parallelizable floor (approx 0.28s) for high-frequency queries.
- Thread Lifecycle Management: Kernel stack allocations and scheduling in pthread_create() and pthread_join() consume measurable time relative to micro-chunk execution windows.

3. Task Granularity and the Hierarchical Concurrency Penalty: Evaluating nested micro-chunks (search_2level_bilingual_and.c, 16 configured workers) resulted in higher latency (0.357s) than balanced single-level 8-thread retrieval (0.295s). This demonstrates that increasing task concurrency beyond physical core capacity without algorithmic pruning increases context-switching overhead and cache thrashing. Furthermore, our selectivity analysis proves that linear search latency is invariant to word rarity, while amortized time-per-match spikes hyperbolically (tau_match> 1,300 mus) for rare terms.

In summary, this work establishes that while multicore Pthreads data partitioning significantly reduces raw retrieval latency, maximum performance gains require pairing parallelism with algorithmic pruning and hardware-conscious workload balancing. Future work will leverage these insights to implement persistent inverted postings lists and dynamic work-stealing task pools, transforming on-demand scanning into sub-millisecond bilingual retrieval.

REFERENCES

[1] A. Kunchukuttan, P. Mehta, and P. Bhattacharyya, "The IIT Bombay English-Hindi Parallel Corpus," in Proc. 11th International Conference on Language Resources and Evaluation (LREC), Miyazaki, Japan, May 2018. Official record: https://aclanthology.org/L18-1548/.

[2] Center for Indian Language Technology, IIT Bombay, "IIT Bombay English-Hindi Corpus," official corpus page, version information and citation guidance. https://www.cfilt.iitb.ac.in/~parallelcorp/iitb_en_hi_parallel/.

[3] S. Jung, D.-J. Chang, and J. W. Park, "Large Scale Document Inversion using a Multi-threaded Computing System," Applied Computing Review, vol. 17, no. 2, pp. 27–35, June 2017. Supplied course reading: P0.pdf.

[4] C. Bonacic, C. García, M. Marín, M. Prieto-Matias, and F. Tirado, "Building Efficient Multi-Threaded Search Nodes," in Proc. 19th ACM Conference on Information and Knowledge Management (CIKM), 2010, DOI: 10.1145/1871437.1871595. Supplied course reading: P1.pdf.

[5] B. Cahoon, K. S. McKinley, and Z. Lu, "Evaluating the Performance of Distributed Architectures for Information Retrieval Using a Variety of Workloads," ACM Transactions on Information Systems, vol. 18, no. 1, pp. 1–43, Jan. 2000, DOI: 10.1145/333135.333136. Supplied course reading: P2.pdf.

[6] G. M. Amdahl, "Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities," in Proc. AFIPS Spring Joint Computer Conference, 1967, pp. 483–485, DOI: 10.1145/1465482.1465560.

[7] The Open Group, "pthread_create," The Open Group Base Specifications, POSIX.1. https://pubs.opengroup.org/onlinepubs/9699919799/functions/pthread_create.html.

[8] The Open Group, "pthread_join," The Open Group Base Specifications, POSIX.1. https://pubs.opengroup.org/onlinepubs/9699919799/functions/pthread_join.html.